

Zero to Hero Study Handbook: `multimodal-pytorch-risk-fusion`

Module 1: Foundations & Architecture

1.1 What this project does

This repository builds a multimodal PyTorch model for CFPB complaints. It combines: - Free-text complaint narrative. - Structured complaint metadata.

It predicts two targets at the same time: - Task A: complaint **Issue** (multi-class, top-15 issues). - Task B: **relief_granted** (binary, derived from **Company response to consumer**).

Primary use cases in this codebase: - Train and compare multimodal vs single-modality baselines (**fusion**, **text**, **tabular**). - Run single-record or batch inference with saved preprocessing and model artifacts. - Benchmark different text encoders under a controlled training regime.

Core project files that define this behavior: - `src/data_loader.py` - `src/models.py` - `src/train.py` - `inference.py` - `compare_encoders.py` - `generate_notebook.py`

1.2 Core paradigms and patterns used here

Definitions first, then where they appear.

1. Multi-task learning A single shared representation feeds multiple output heads.
 - In this repo: `MultiModalRiskNet` has `head_a` and `head_b` in `src/models.py`.
 - In training: total loss is `loss_a + cfg.aux_weight * loss_b` in `train_multitask()` (`src/train.py`).
2. Multimodal fusion A model merges multiple input modalities (text + tabular) into one representation.
 - In this repo: `GatedFusion` combines `h_text` and `h_struct` in `src/models.py`.
3. Learned gating A sigmoid gate learns per-example modality weighting.
 - In this repo: `g = sigmoid(W * [h_text; h_struct])`, then `fused = g * h_text + (1 - g) * h_struct` in `GatedFusion.forward()`.
4. Ablation-driven evaluation Train controlled variants where one component is removed to measure contribution.
 - In this repo: `mode in ("fusion", "text", "tabular")` in `MultiModalRiskNet`.
 - Notebook and scripts train all three modes for apples-to-apples comparison.

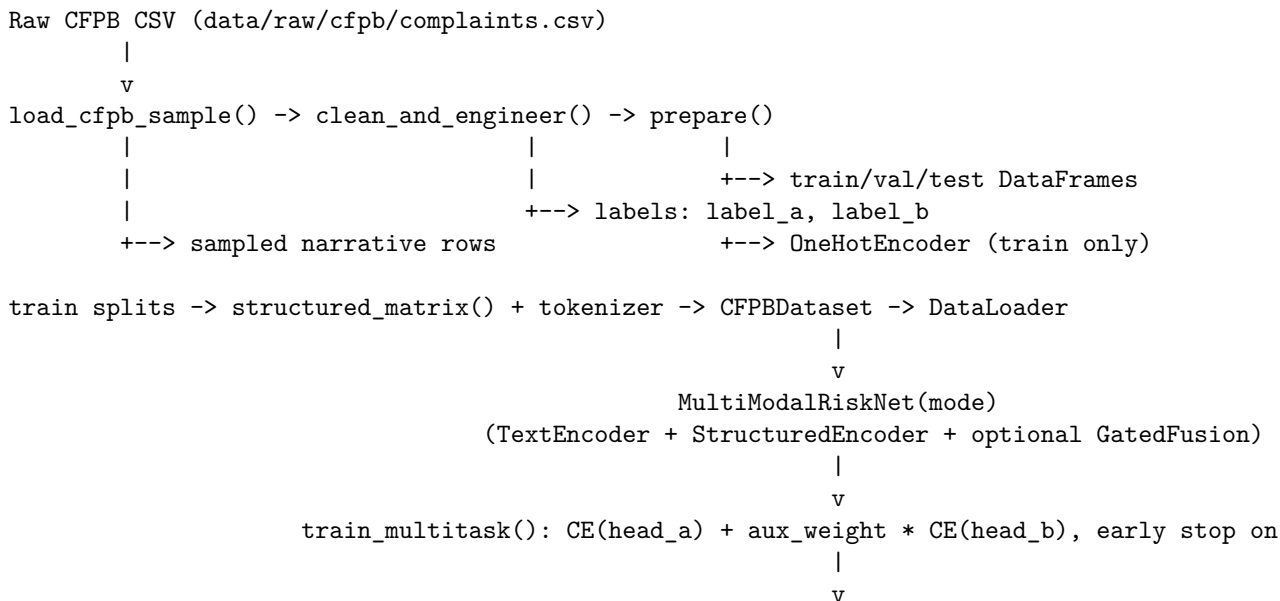
5. Leakage-aware tabular preprocessing Prevent target leakage by ensuring only submission-time fields become features.
 - In this repo: post-outcome fields are not used in structured features; CAT_FEATURES is explicit in `src/data_loader.py`.
6. Train/val/test discipline with train-only fitting Preprocessing is fit on train split only and reused on val/test.
 - In this repo: `OneHotEncoder.fit(train[CAT_FEATURES])` in `prepare()`.
7. Inference parity pattern Inference reuses saved preprocessing artifacts from training.
 - In this repo: `inference.py` loads `preprocess.pkl` and applies `_engineer_structured()` with saved caps and encoder.

1.3 Architecture and component interactions

High-level components:

- Data pipeline (`src/data_loader.py`): sampling, cleaning, feature engineering, splitting, encoding, datasets.
- Model stack (`src/models.py`): text encoder, structured encoder, gated fusion, two heads.
- Training loop (`src/train.py`): AMP policy, class-weighted losses, scheduler, early stopping, metrics.
- Notebook generator (`generate_notebook.py`): produces executable notebook narrative.
- Inference runtime (`inference.py`): loads saved artifacts and emits predictions.
- Encoder benchmark (`compare_encoders.py`): controlled backbone comparison.

Main training/inference flow diagram:



```

predict()/compute_metrics() -> figures/*.csv and checkpoint artifi
|
+--> checkpoints/fusion_best.pt
+--> checkpoints/preprocess.pkl

```

Inference path:
input row(s)/CSV -> _engineer_structured() + tokenizer -> model.forward()
-> issue_top + relief_prob + text_reliance

Module 2: Repository Map

2.1 High-priority file map for new contributors

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
pyproject.toml	Project metadata and dependency declaration for uv	N/A	requires-python = ">=3.12.10", dependencies, custom torch index
README.md	User-facing workflow, dataset source, run commands, architecture summary	N/A	pytorch-cu128 Setup commands, inference CLI examples, layout contract
src/data_loader.py	End-to-end data preparation and dataset objects	load_cfpb_sample, CSV_PATH, clean_and_engineer, SECOLS, prepare, PRODUCT_MAP, structured_matrix, RELIEF_POSITIVE, CFPBDataset, CAT_FEATURES, make_collate, class_weights	
src/models.py	Multimodal architecture and ablation modes	masked_mean, mode in TextEncoder, ("fusion","text","tabular"), StructuredEncoder, fusion_dim, GatedFusion, dropout, MultiModalRiskNet, trust_remote_code, freeze_text_backbone	

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>src/train.py</code>	Training loop, AMP policy, validation metrics	<code>set_seed</code> , <code>amp_policy</code> , <code>TrainConfig</code> , <code>predict</code> , <code>compute_metrics</code> , <code>train_multitask</code>	<code>TrainConfig</code> fields: <code>epochs</code> , <code>lr</code> , <code>aux_weight</code> , <code>patience</code> , <code>head_b_metric</code> , etc.
<code>inference.py</code>	Production-style prediction entrypoint for single/batch inputs	<code>_engineer_structured_load</code> , <code>predict</code> , <code>main</code>	<code>add_args</code> : <code>--narrative/--input</code> , <code>--model-dir</code> , <code>--top-k</code> , <code>--device</code> , <code>--output</code>
<code>generate_notebook.py</code>	Programmatically builds <code>multimodal_pytorch_risk_fusion.ipynb</code> from cells	<code>md</code> , <code>code</code> (cell constructors)	Writes notebook file; notebook code saves model/preprocess artifacts
<code>compare_encoders.py</code>	Controlled text encoder bake-off under same data/splits	<code>wait_for_gpu</code> , <code>build_loaders</code> , <code>train_one</code> , <code>main</code>	<code>ENCODERS</code> map, CLI: <code>--encoders</code> , <code>--unfreeze</code> , <code>--max-len</code> , <code>--min-free-gb</code>
<code>multimodal_pytorch_risk_fusion.ipynb</code>	Narrative execution surface for full experiment workflow	Notebook cells call <code>src/*</code> modules	<code>MAX_LEN</code> , checkpoint save cell, abla- tion/calibration slice analysis
<code>figures/ablation.Saved</code>	ablation metrics from notebook pipeline	N/A	Columns include Task A/B metrics by model mode
<code>figures/encoder_comparison.csv</code>	Encoder benchmark outputs	N/A	<code>encoder</code> , <code>mode</code> , <code>A_macro_f1</code> , <code>A_accuracy</code> , <code>B_auroc</code> , etc.
<code>figures/encoder_report.html</code>	Report and analysis of encoder benchmark outcomes	N/A	Documents run regime and interpretation

checkpoints/preprocessed.pkl	Preprocessed preprocessing state for inference parity	N/A (artifact consumed by <code>inference.py</code>)	Expected keys include <code>encoder</code> , <code>company_topk</code> , <code>subproduct_topk</code> , <code>cat_features</code>
------------------------------	--	---	---

2.2 What to read first

If you only have one pass: 1. `src/data_loader.py` 2. `src/models.py`
3. `src/train.py` 4. `inference.py` 5. `generate_notebook.py` 6.
`compare_encoders.py`

This order matches the runtime lifecycle: data -> model -> training -> inference
-> orchestration/experiments.

Module 3: Core Execution Flows

3.1 Flow A: Training and artifact creation

This is the main learning flow of the repository (driven from the generated notebook).

Step 1: Load and sample raw data - Function: `load_cfpb_sample(path=CSV_PATH, n_target=..., seed=..., chunksize=..., narr_total_est=...)`. - Reads only USECOLS columns from CSV in chunks (`pd.read_csv(..., chunksize=...)`). - Keeps rows where Consumer complaint narrative is present and non-empty. - Returns sampled `pd.DataFrame`.

Step 2: Clean and engineer labels/features - Function: `clean_and_engineer(df, top_issues=15)`. - Adds canonical text field `narrative`. - Canonicalizes product via `PRODUCT_MAP`. - Adds structured columns: `product`, `sub_product`, `company`, `state`, `tags`, `year`, `month`, `dow`. - Builds Task B label: `label_b = 1` iff Company response to consumer is in `RELIEF_POSITIVE`. - Filters Task A to top-N issues.

Step 3: Split and fit structured encoder - Function: `prepare(df, seed=42, top_companies=100, top_subproducts=40, test_size=0.15, val_size=0.15)`. - Creates `label_a` by issue-to-index map. - Splits into train/val/test with stratification on `label_a`. - Caps `company` and `sub_product` to train-derived top-K sets; others -> `OTHER`. - Fits `OneHotEncoder` on `train[CAT_FEATURES]` only. - Returns `PreparedData` dataclass with: - `train`, `val`, `test` `DataFrames`. - `encoder` (`OneHotEncoder`). - `issue_classes` list. - `struct_dim` int. - `company_topk`, `subproduct_topk` sets.

Step 4: Build dataset and loaders - `structured_matrix(df, encoder)` creates numpy matrix for tabular inputs. - `CFPBDataset` stores: - `input_ids`, `attention_mask` from tokenizer output. - structured float tensor. - `label_a`,

label_b long tensors. - `make_collate(tokenizer)` pads text dynamically and stacks structured/labels.

Expected batch dictionary shape at runtime:

```
{
    "input_ids": Tensor[batch, seq_len],
    "attention_mask": Tensor[batch, seq_len],
    "structured": Tensor[batch, struct_dim],
    "label_a": Tensor[batch],
    "label_b": Tensor[batch],
}
```

Step 5: Build model - Class: `MultiModalRiskNet(struct_dim, num_classes_a, num_classes_b, model_name=..., fusion_dim=256, dropout=..., mode=...)`. - `mode` controls architecture variant: - `fusion`: text + structured + gated fusion. - `text`: text only. - `tabular`: structured only. - Optional efficiency method: `freeze_text_backbone(n_trainable_top_layers=2)`.

Step 6: Train - Function: `train_multitask(model, train_loader, val_loader, device, cfg, class_weights_a, class_weights_b, verbose=True)`. - Uses: - Mixed precision policy from `amp_policy(device)`. - AdamW optimizer with weight decay grouping. - Linear warmup scheduler from transformers. - Gradient clipping (`cfg.max_grad_norm`). - Loss per step:

```
loss_a = cross_entropy(logits_a, label_a, weight=cw_a)
loss_b = cross_entropy(logits_b, label_b, weight=cw_b)
loss = loss_a + cfg.aux_weight * loss_b
```

- Early stopping signal: validation `A_macro_f1`.

Step 7: Evaluate and export outputs - `predict(model, loader, ...)` returns:

```
{
    "prob_a": np.ndarray[N, num_a],
    "prob_b": np.ndarray[N, 2],
    "y_a": np.ndarray[N],
    "y_b": np.ndarray[N],
    "gate": np.ndarray[N] | None,
}
```

- `compute_metrics(res, head_b_metric="auroc")` computes:
 - Task A: `A_accuracy`, `A_macro_f1`, `A_weighted_f1`.
 - Task B: `B_auroc` (if binary), `B_macro_f1`.

Step 8: Save training artifacts (from notebook code in `generate_notebook.py`) - `torch.save({...}, checkpoints/fusion_best.pt)` with keys: - `model_state_dict`, `struct_dim`, `num_a`, `max_len`, `issue_classes`. - `pickle.dump({...}, checkpoints/preprocess.pkl)` with keys: - `encoder`, `issue_classes`, `company_topk`, `subproduct_topk`, `cat_features`.

3.2 Flow B: Inference CLI (`inference.py`)

Step 1: Parse input mode - Mutually exclusive CLI args: `--narrative` (single record) OR `--input` (CSV).

Step 2: Load artifacts - `load(model_dir, device)` expects both: - `fusion_best.pt` - `preprocess.pkl` - Instantiates `MultiModalRiskNet(..., mode="fusion")` and loads weights. - Loads tokenizer `distilbert-base-uncased`.

Step 3: Rebuild structured features exactly as training - `_engineer_structured(df, pre)`: - Maps product via `PRODUCT_MAP`. - Builds date parts (`year`, `month`, `dow`) from `Date received`. - Applies top-K caps via `pre["company_topk"]` and `pre["subproduct_topk"]`. - Structured matrix computed as: - `pre["encoder"].transform(engineered_df[pre["cat_features"]])`.

Step 4: Predict - `predict(df, model, tok, pre, ckpt, device, max_len=128, batch_size=32, top_k=3)`: - Tokenizes narratives. - Builds model batch dict with `input_ids`, `attention_mask`, `structured`. - Runs forward pass. - Produces per-row result object:

```
{
    "issue_top": List[Tuple[str, float]],
    "relief_prob": float,
    "text_reliance": float | None,
}
```

Step 5: Output - Always prints human-readable predictions to stdout. - If `--output` is set, writes CSV columns: - `predicted_issue` - `issue_confidence` - `relief_prob` - `text_reliance`

3.3 Flow C: Encoder comparison experiment (`compare_encoders.py`)

Purpose: - Hold data/splits/training loop constant while swapping text backbone.

Key mechanics: - `ENCODERS` map defines candidate model IDs and `trust_remote_code` flags. - `wait_for_gpu()` polls free VRAM and selects device and batch size. - `train_one()` trains one (`mode`, `encoder`) pair and returns metric dict. - Main flow trains: - `tabular` once. - `text` and `fusion` for each selected encoder. - Writes comparison CSV to: - `figures/encoder_comparison_deep.csv` (full FT regime) - or `figures/encoder_comparison.csv`.

Output row schema (DataFrame columns): - `encoder`, `mode`, `A_macro_f1`, `A_weighted_f1`, `A_accuracy`, `B_auroc`, `B_macro_f1`, `minutes`.

Module 4: Setup & Run Guide

4.1 Prerequisites

- OS: Linux-friendly workflow (repo itself is cross-platform Python).

- Python: 3.12.10 (from `.python-version` and `pyproject.toml`).
- Package manager/environment: `uv`.

4.2 Install dependencies

```
cd /path/to/multimodal-pytorch-risk-fusion
uv sync
```

Dependency source of truth: - `pyproject.toml` (direct deps). - `uv.lock` (resolved versions).

4.3 Data setup

Expected training CSV path in code: - `data/raw/cfpb/complaints.csv` (`CSV_PATH` in `src/data_loader.py`).

README-provided commands:

```
mkdir -p data/raw/cfpb
wget -O data/raw/cfpb/complaints.csv.zip https://files.consumerfinance.gov/ccdb/complaints.csv.zip
unzip data/raw/cfpb/complaints.csv.zip -d data/raw/cfpb
```

4.4 Generate and run notebook workflow

Generate notebook file from Python script:

```
uv run python generate_notebook.py
```

Then execute notebook (interactive or headless):

```
uv run jupyter lab multimodal_pytorch_risk_fusion.ipynb
uv run jupyter nbconvert --to notebook --execute --inplace multimodal_pytorch_risk_fusion.ipynb
```

Expected artifacts produced by notebook code: - `checkpoints/fusion_best.pt`
- `checkpoints/preprocess.pkl` - `figures/ablation.csv` and generated PNG plots.

4.5 Inference commands

Single complaint:

```
uv run python inference.py \
  --narrative "I was charged a late fee even though I paid on time." \
  --product "Credit card or prepaid card" \
  --company "BIG BANK" \
  --state CA
```

Batch mode:

```
uv run python inference.py --input complaints.csv --output preds.csv
```


4.6 Encoder benchmark command

```
uv run python compare_encoders.py --encoders distilbert,modernbert,neobert,mrbert --unfreeze
```

4.7 Environment variables and config files

Required environment variables from repository code: - None explicitly required by project scripts.

Optional ecosystem variable you may use: - `HF_TOKEN` can improve Hugging Face Hub rate limits (warning appears in notebook output), but it is not directly read in project code.

Configuration files used: - `pyproject.toml` (dependencies/project metadata). - `uv.lock` (locked transitive versions). - `.python-version` (Python version pin).

4.8 Migration/seeding/external service steps

- Database migration: none (no DB layer in repo).
- Seeding script: none.
- External service bootstrap: none required for core local workflow beyond downloading CFPB CSV and model downloads from Hugging Face.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan for a new learner

Phase 1: Understand the problem and outputs 1. Read `README.md` sections on task framing, architecture, setup, and ablations. 2. Read `inference.py` top-to-bottom to see final user-facing inputs/outputs.

Phase 2: Master the data contract 3. Read `src/data_loader.py` in order: - constants (`USECOLS`, `PRODUCT_MAP`, `CAT_FEATURES`), - sampling, - label engineering, - split/encoding, - dataset/collate.

Phase 3: Master model internals 4. Read `src/models.py`: - `masked_mean`, - `TextEncoder` and `StructuredEncoder`, - `GatedFusion`, - `MultiModalRiskNet` and mode behavior, - `freeze_text_backbone`.

Phase 4: Master optimization/evaluation logic 5. Read `src/train.py`: - `TrainConfig`, - `amp_policy`, - `train_multitask`, - `predict` and `compute_metrics`.

Phase 5: Orchestration and experiments 6. Read `generate_notebook.py` to understand full narrative execution and artifact saving. 7. Read `compare_encoders.py` for controlled experiment design. 8. Inspect `figures/*.csv` and `figures/encoder_report.md` to connect code to measured outcomes.

5.2 Practice exercises (with model solution outlines)

Exercise 1 Question: Which exact columns are read from the raw CFPB CSV, and why is this important?

Solution outline: - In `src/data_loader.py`, `USECOLS` defines the read schema. - This constrains I/O, reduces memory pressure, and documents the minimum required raw fields.

Exercise 2 Question: How is leakage controlled for Task B while still deriving `label_b` from response data?

Solution outline: - `label_b` is derived in `clean_and_engineer()` from `Company response to consumer`. - Structured features are only `CAT_FEATURES`, which exclude response/timeliness/public response/date sent fields. - So label derivation uses response, but model inputs do not include post-outcome response fields.

Exercise 3 Question: What does `prepare()` return, and which pieces are required later by inference?

Solution outline: - `PreparedData` returns splits, fitted encoder, issue classes, struct dimension, and top-K caps. - Inference needs the persisted analogs: encoder, issue classes, company/subproduct top-K sets, cat feature order.

Exercise 4 Question: In `MultiModalRiskNet`, what changes when mode is `fusion` vs `text` vs `tabular`?

Solution outline: - `text`: only `TextEncoder`; no gate. - `tabular`: only `StructuredEncoder`; no gate. - `fusion`: both encoders + `GatedFusion`; returns gate for interpretability.

Exercise 5 Question: Write the exact multi-task loss formula used by training and identify where the weighting comes from.

Solution outline: - Formula: `loss = CE_A + aux_weight * CE_B`. - `aux_weight` comes from `TrainConfig (aux_weight: float = 0.3)`. - Class weights come from `class_weights()` and are passed to `F.cross_entropy` for each head.

Exercise 6 Question: What determines early stopping and where is it implemented?

Solution outline: - Early stop monitor is validation `A_macro_f1` in `train_multitask()`. - `cfg.patience` controls allowed non-improving epochs before break.

Exercise 7 Question: What is the exact prediction object shape returned by `inference.py` before optional CSV export?

Solution outline: - Each item contains: - `issue_top`: top-k list of (`issue_name`, `probability`) pairs, - `relief_prob`: float, - `text_reliance`: float or None.

Exercise 8 Question: Why does `inference.py` call `_engineer_structured()` instead of directly using input columns as-is?

Solution outline: - It reproduces training-time feature engineering: product mapping, null handling, date decomposition, top-K capping. - Without this parity, inference feature distribution/order would drift from training.

Exercise 9 Question: How does `compare_encoders.py` keep the comparison fair across encoders?

Solution outline: - Same sampled data and split process, same training function (`train_one` -> `train_multitask`), same metric extraction. - Tabular-only baseline trained once because it is encoder-independent.

Exercise 10 Question: List the required artifact files and their key content for successful inference.

Solution outline: - `checkpoints/fusion_best.pt`: model weights + `struct_dim` + `num_a` + `max_len` + `issue_classes`. - `checkpoints/preprocess.pkl`: fitted one-hot encoder + category caps + categorical feature order.

Learner Verification Checklist

Use this checklist after studying all modules.

- Can you explain why the project uses two tasks and how the shared trunk supports both?
- Can you trace the training data path from raw CSV to a `CFPBDataset` batch dictionary?
- Can you justify why `CAT_FEATURES` is leakage-safe for the defined targets?
- Can you explain how `GatedFusion` computes and applies the text/structured blend?
- Can you describe what changes across `fusion`, `text`, and `tabular` modes?
- Can you explain how class weights are computed and where they enter the loss?
- Can you describe the early stopping rule and what metric it monitors?
- Can you list every field saved in checkpoint/preprocess artifacts and why each is needed?
- Can you explain the inference parity guarantees implemented in `_engineer_structured()`?
- Can you explain how encoder benchmarking is structured to support fair comparison?

If you can answer all ten clearly with file-level references, you have reached working mastery of this repository.